浙江大学

本科实验报告

课程名称：	计算机组成与设计
姓　　名：	吴溪语
学　　院：	信息与电子工程学院
系：	信电
专　　业：	电子科学与技术
学　　号：	[redacted]
指导教师：	屈民军、唐奕

2025 年 10 月 8 日

专业：电子科学与技术
姓名：吴溪语
学号：[redacted]
日期：2025.10.8
地点：高分子楼 201

浙江大学实验报告

课程名称：计算机组成与设计　指导老师：屈民军、唐奕　成绩：

实验名称：Lab8 快速加法器的设计　实验类型：设计实验

一、实验目的

（1）掌握快速加法器的设计。
（2）熟悉流水线技术。
（3）掌握时序仿真的工作流程。

二、实验任务与要求

（1）采用“进位选择加法”技术设计 32 位加法器，并对设计进行功能仿真和时序仿真。
（2）采用四级流水线技术设计 32 位加法器，并对设计进行功能仿真和时序仿真。

三、实验原理

1.四位先行进位加法器的设计

两个加数分别为 $A_3A_2A_1A_0$ 和 $B_3B_2B_1B_0$， C_{-1} 为最低位进位。设两个辅助变量分别为 $G_3G_2G_1G_0$ 和 $P_3P_2P_1P_0$: $G_i = A_i \,\&\, B_i$， $P_i = A_i + B_i$。

一位全加器的逻辑表达式可转化为

$$\begin{cases} S_i = P_i\overline{G_i} \oplus C_{i-1} \\ C_i = G_i + P_iC_{i-1} \end{cases}$$

利用上述关系，一个四位加法器的进位计算就变转化为

$$\begin{cases} C_0 = G_0 + P_0C_{-1} \\ C_1 = G_1 + P_1C_0 = G_1 + P_1G_0 + P_1P_0C_{-1} \\ C_2 = G_2 + P_2C_1 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_{-1} \\ C_3 = G_3 + P_3C_2 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_{-1} \end{cases}$$

由此可以看出，每一个进位的计算都直接依赖于整个加法器的最初输入，而不需要等待相邻低位的进位传递。理论上，每一个进位的计算都只需要三个门延迟时间，即产生 $G[i]$、

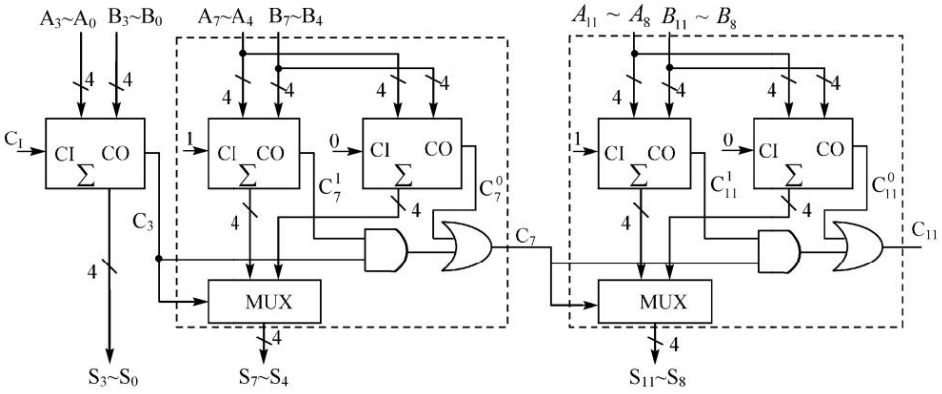
$P[i]$的与门和或门，输入为$G[i]$、$P[i]$、C_{-1}的与门，以及最终的或门。同样道理，理论上最终结果 sum 的得到只需要四个门延迟时间。

实际上，当加数位数较大时，输入需要驱动的门数较多，其 VSI 实现的输出时延增加很多，考虑到互连线的延时情况将会更加糟糕。因此，通常在芯片实现时先设计位数较少的超前进位加法器结构，而后以此为基本结构来构造位数较大的加法器。

2.进位选择加法器结构

借鉴并行计算的思想,人们提出了进位选择加法器结构,或者称为有条件的加法器结构,其算法的实质是增加硬件面积换取速度性能的提高。二进制加法的特点是进位或者为逻辑 1 或者为逻辑 0，二者必居其一。将进位链较长的加法器分为 M 块，分别进行加法计算，对除去最低位计算块外的 M-1 块加法结构复制两份，其进位输入分别预定为逻辑 1 和逻辑 0 于是，M 块加法器可以同时并行进行各自的加法运算，然后根据各自相邻地位加法运算结果产生的进位输出，选择正确的加法结果输出。

下图所示为 12 位进位选择加法器的逻辑结构图:



12 位加法器划分为 3 块，最低一块(4 位)由 4 位超前进位加法器直接构成，后两块分别假设前一块的进位为 0 或 1 将两种结果都计算出来，再根据前级进位选择正确的和与进位。如果每一块加法结构内部都采用速度较快的超前进位加法器结构，那么进位选择加法器的计算时延为

$$t_{CSA} = t_{carry} + (M-2)t_{MUX} + t_{sum}$$

其中，t_{sum}、t_{carry}分别为加法器的和与加法器的进位时延，t_{MUX}为数据选择器的时延。

3.流水线加法的设计

在数字系统中，如果完成某些复杂逻辑功能需要较长的时延，就会使系统很难运行在较高的工作频率上。流水线设计(pipeline design)是经常用于提高系统运行速度的一种有效方法，

即在长时延的逻辑功能块中插入触发器，将复杂的逻辑操作分步进行，减少每个部分的操作，从而提高系统的运行速度。

如图所示为采用 4 级流水线技术的 32 位加法器的原理框图，采用了 5 级锁存、4 级 8 位加法，整个加法只受 8 位加法器工作速度了限制，从而提高了整个加法器工作速度。

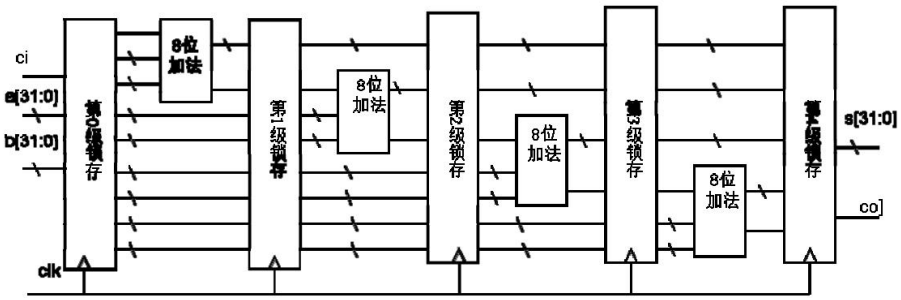


图 5.7　采用 4 级流水线技术的 32 位加法器原理框图

四、主要设备仪器

安装 Vivado 与 ModelSimSE 的计算机。

五、实验内容

1.编写 4 位先行进位加法器的 Verilog HDL 代码，并用 ModelSim 软件进行功能仿真；

2.32 位进位选择加法器的设计：

（1）编写用 32 位进位选择加法器的 Verilog HDL 代码，并用 ModelSim 软件进行功能仿真；

（2）对 32 位加法器进行时序仿真。其步骤如下：

（a）建立 Vivado 工程，除了加入设计文件，还需要加入仿真测试文件，同时设置其属性为 Simulation Only；

（b）对 32 位进位选择加法器进行综合、实现，注意本例不需要约束；

（c）在工程界面的 Flow Navigator 窗口中，右击 SIMULATION，在弹出的快捷菜单中选择 Simulation Setting…命令，将仿真器选择为 ModelSim Simulator，同时加入仿真测试文件，指定仿真库存储路径，单击 OK 按钮；

（d）在工程界面的 FLOW Navigator 窗口中，单击 SIMULATION->Run Simulation 选项，在随后弹出的快捷菜单中选择 Run Post-Implementation Timing Simulation 命令，即可启动时序仿真。

3.编写采用 4 级流水线技术的 32 位加法器的 Verilog HDL 代码，并对设计进行时序仿真。

六、实验结果与仿真分析

（1）4 位先行进位加法器的设计

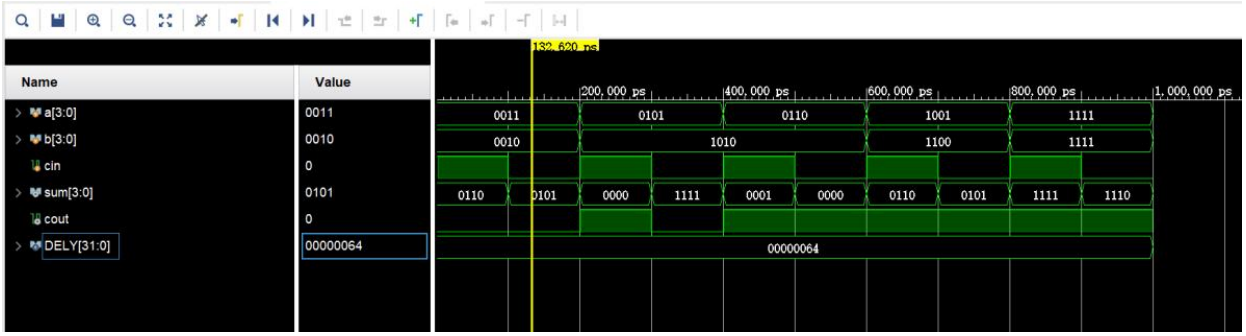
①设计代码：

```
module adder_4bits(
    input [3:0]a,b,//输入
    input cin,//最低位的进位
    output [3:0]s,//各级的和
    output cout//最高位的进位
    );

    wire [2:0]c;//各级进位
    wire [3:0]p,g;//辅助变量

    //辅助变量计算
    assign p=a|b;//按位或
    assign g=a&b;//按位与

   //进位计算
    assign c[0]=g[0]||p[0]&cin;
    assign c[1]=g[1]||p[1]&c[0];
    assign c[2]=g[2]||p[2]&c[1];
    assign cout=g[3]||p[3]&c[2];

    //和计算
    assign s[0]=(p[0]&~g[0])^cin;
    assign s[1]=(p[1]&~g[1])^c[0];
    assign s[2]=(p[2]&~g[2])^c[1];
    assign s[3]=(p[3]&~g[3])^c[2];

endmodule
```

②仿真结果与分析

如下图所示，在不同的 a，b 和 cin 下，sum 与 cout 的结果均正确。例如，输入 a=1001 与 b=1100 且 cin=1 时，输出 sum 为 0110 且 cout 为 1，符合二进制中 1001+1100+1=10110。说明 4 位先

行进位加法器的功能设计符合预期结果。



（2）4bit 进位选择加法器的设计

为了支持后续 32bit 进位选择加法器的设计，需要先设计一个 4bit 进位选择加法器模块。

①代码：

```
module sel_adder_4bits(
    input [3:0]a,b,
    input cin,
    output [3:0]sum,
    output cout
    );

    wire [3:0] sum0,sum1;//定义不同情况下的输出

    //预计算不同 cin 下的结果
    adder_4bits cla0(.a(a), .b(b), .cin(1'b0), .s(sum0), .cout(cout0)); // cin=0 时的进位输出 cout0
    adder_4bits cla1(.a(a), .b(b), .cin(1'b1), .s(sum1), .cout(cout1)); // cin=1 时的进位输出 cout1

    //  根据实际 cin 选择最终 cout 和 sum
    assign cout=cin?cout1:cout0;
    assign sum=cin?sum1:sum0;

endmodule
```

（3）32bit 进位选择加法器的设计

①设计思路：

参考设计原理中的 12bit 加法器，可以将 32bit 进位选择加法器划分为 8 块，最低一块(4 位)

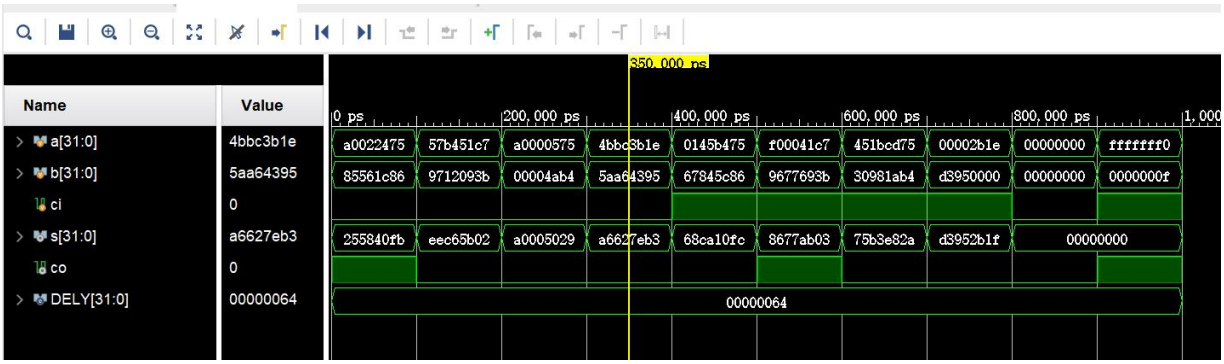
由 4 位超前进位加法器直接构成，后 7 块分别假设前一块的进位为 0 或 1 将两种结果都计算出来，再根据前级进位选择正确的和与进位。此模块由顶层模块 adder_32bits，4 位先行进位加法器 adder_4bits 和 4 位进位选择加法器 sel_adder_4bits 构成。

②顶层模块代码：

```
module adder_32bits(
    input [31:0]a,b,//输入
    input cin,//最低位进位
    output [31:0]s,//和
    output cout//最高位进位
    );
    wire[6:0]c;//定义各块的进位

    //最低位的计算模块
    adder_4bits mod1(.a(a[3:0]),.b(b[3:0]),.cin(cin),.s(s[3:0]),.cout(c[0]));

    //其余 7 个计算模块
    adder_4bits mod2(.a(a[7:4]),.b(b[7:4]),.cin(c[0]),.s(s[7:4]),.cout(c[1]));
    adder_4bits mod3(.a(a[11:8]),.b(b[11:8]),.cin(c[1]),.s(s[11:8]),.cout(c[2]));
    adder_4bits mod4(.a(a[15:12]),.b(b[15:12]),.cin(c[2]),.s(s[15:12]),.cout(c[3]));
    adder_4bits mod5(.a(a[19:16]),.b(b[19:16]),.cin(c[3]),.s(s[19:16]),.cout(c[4]));
    adder_4bits mod6(.a(a[23:20]),.b(b[23:20]),.cin(c[4]),.s(s[23:20]),.cout(c[5]));
    adder_4bits mod7(.a(a[27:24]),.b(b[27:24]),.cin(c[5]),.s(s[27:24]),.cout(c[6]));
    adder_4bits mod8(.a(a[31:28]),.b(b[31:28]),.cin(c[6]),.s(s[31:28]),.cout(cout));

endmodule
```

③功能仿真结果与分析：

下图为 simulation 结果图，仿真结果与实际运算结果相同，说明 32bit 进位选择加法器的功能设计符合预期。



④时序仿真结果与分析

时序仿真结果显示，运算结果存在一定的延时，输出结果的变化慢于输入的变化。另外，在 co 的仿真结果中存在尖峰，体现了 1 型的竞争冒险。这是由于在组合逻辑中，当输入信号发生变化时，由于门电路的延迟，不同路径上的信号到达输出端的时间有差异，导致逻辑运算的结果在短时间内出现错误。

七、实验中碰到的问题与解决方法

在这个实验中，由于对 verilog 语法的不熟练，我最初把 adder_4bits 中的辅助变量计算弄错了，将 assign p=a|b 写成了 assign p=a||b，也就是把按位或与逻辑或弄错了，导致最终的结果有误。在反复检查之后发现了这一问题，查阅了相关资料，弄清楚了在这个实验中按位或的运用是为了判断每一位是否能将低位的进位传递到高位，而逻辑或是对整体进行运算，修正了代码，最终获得了正确的结果。

八、思考题

1.为什么要进行时序仿真？

时序仿真相比功能仿真更贴近电路实际工作状态，可以加载布局布线后器件的模块与连线延时信息。例如本实验中 32 位选择进位加法器的计算时延由进位时延、加和时延、数据选择器时延等构成，时序仿真可判断其结果稳定输出时间是否充足，还能以此为依据确定加法器正常工作的时钟频率范围，有效排查时延、竞争冒险等问题，让仿真结果更贴合芯片实际运

行情况。

2.采用流水线技术有什么优缺点？

优点：①大幅提升系统吞吐量。将长延迟任务拆分为多个短步骤，各步骤并行处理，在连续输入数据的情况下，单位时间内可完成更多任务。②降低单步逻辑复杂度。拆分后的每个流水线级逻辑更简单，单个步骤的延迟缩短，支持更高的工作时钟频率。③资源利用率更均衡。避免单个复杂模块长时间占用资源。

缺点：①增加硬件开销，需额外触发器存储各级中间数据，占用更多芯片面积，提升设计成本。②单条数据处理延迟增加，虽提升吞吐率，但单条数据需经过多段流水线，总延迟比非流水线结构更长。③可能引入数据冒险、控制冒险、结构冒险等问题。

九、实验总结

通过本次快速加法器的设计实验，对数字系统设计的核心思想有了更深刻的认识，并重新熟悉了 verilog 语言，收获颇丰。

在设计 4 位先行进位加法器、32 位进位选择加法器的过程中，我理解了“以面积换速度”的设计思想。先行进位加法器通过提前计算进位，避免了行波进位加法器中进位逐级传递的延迟，而进位选择加法器更是进一步利用并行计算和选择机制，将 32 位加法器划分为多个子块，同时预计算不同进位假设下的结果，再根据实际进位选择正确输出，极大提升了加法运算速度。

在编写代码时，我从最开始将按位或（|）误写为逻辑或（||），导致结果错误，到后来通过查阅资料、反复检查修正错误，我认识到 Verilog 语法细节的重要性，也更加重视代码编写的严谨性。仿真与验证也是这个实验中非常重要的部分。功能仿真验证了电路逻辑的正确性，而时序仿真则让我看到了实际电路中的延迟和竞争冒险现象，比如 32 位进位选择加法器进位输出（cout）出现的尖峰，这让我明白理论设计与实际硬件实现之间存在差距，必须考虑实际的硬件特性。另外，通过分析仿真结果，我也掌握了如何排查综合与仿真过程中出现的错误的方法，大大提升了我工程实践与分析能力。